FID
Première Année

المدرسة الوطنية للعلوم التطبيقية – أكادير
ⵜⵉⵎⵙⵙⵓⵜ ⵜⵜⴰⵎⵓⵔⵜ ⵏ ⵜⵣⵔⴼⵜ ⵜⵜⴰⵏⵎⵓⵔⵜ - ⴰⵅⴰⴷⵉⵔ
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES - AGADIR

RAPPORT :
PROJET D'ANALYSE DE DONNÉES

Préparé par:

- Erraoui Reda

Encadré par :

- Pr.El Asri Brahim

Clustering K-means et Classification Hiérarchique

Année universitaire:
2024/2025

Date : 30 avril 2025

List of Figures

II.1 Étapes du processus de l'algorithme K-Means 5
II.2 Méthode du coude pour déterminer le nombre optimal de clusters. 7

III.1 Illustration du processus de fusion progressive en CAH 8

IV.1 Page d'accueil de l'application. L'utilisateur peut importer un fichier Excel ou CSV. . 11
IV.2 Visualisation des clusters obtenus avec l'algorithme K-Means. 12
IV.3 Affichage de la courbe de convergence montrant l'inertie au fil des itérations. 13
IV.4 Dendrogramme illustrant les fusions successives entre clusters (CAH). 14
IV.5 Options d'exportation vers PDF ou ZIP des graphiques et logs. 15
IV.6 Page d'export des résultats avec options PDF et ZIP. 16

Table des matières

I Packages 4

II K-Means 5

II.1 Introduction . . . 5
II.2 Étapes de l'algorithme . . . 5
II.3 Formulation mathématique . . . 6
II.4 Distances possibles . . . 6
II.5 Critères de convergence . . . 6
II.6 Initialisation K-Means++ . . . 6
II.7 Détermination du nombre optimal de clusters . . . 7
II.8 Avantages . . . 7
II.9 Inconvénients . . . 7

III Classification Hiérarchique Ascendante 8

III.1 Introduction . . . 8
III.2 Principe de l'algorithme . . . 8
III.3 Description de l'algorithme . . . 8
III.4 Distances entre individus . . . 9
III.5 Méthodes de liaison (linkage) . . . 9
III.6 Dendrogramme . . . 10
III.7 Détermination du nombre de clusters . . . 10
III.8 Avantages . . . 10
III.9 Inconvénients . . . 10

IV Interface Graphique 11

IV.1 Page d'Accueil . . . 11
IV.2 Page de Visualisation des Résultats (K-Means) . . . 12
IV.3 Courbe de convergence . . . 13
IV.4 Visualisation Hiérarchique (CAH) . . . 14
IV.5 Page de Visualisation du Dendogramme . . . 15
IV.6 Export des Résultats . . . 16

V Conclusion 17

VI Annexe 18

I Packages

Dans ce projet, plusieurs bibliothèques Python ont été utilisées pour la mise en œuvre des algorithmes de clustering, l'analyse de données et la création d'une interface graphique interactive. Voici les principaux packages :

Packages Python

- **pandas:** Manipulation de données tabulaires, lecture des fichiers CSV/Excel.
- **numpy:** Calculs numériques : distances, moyennes, inertie, etc.
- **matplotlib.pyplot**: Tracé des graphes (courbes, figures statiques, dendrogrammes).
- **scipy.cluster.hierarchy:** Génération de la CAH et visualisation du dendrogramme.
- **scipy.spatial.distance:** Calcul des matrices de distances pour la CAH.
- **sklearn.decomposition:** Réduction dimensionnelle pour la visualisation des clusters.
- **plotly.graph_objects:** Tracé dynamique et interactif de la courbe de convergence (inertie).
- **streamlit:** Création de l'interface graphique web interactive.
- **json:** Chargement de fichiers '.json' (animations lottie).
- **base64:** Encodage d'images et fichiers en base64 (pour les fonds Streamlit et ZIP).
- **os:** Manipulation du système de fichiers : répertoires, enregistrements.
- **zipfile:** Création d'archives ZIP des résultats.
- **io:** Manipulation de fichiers en mémoire (buffer pour les PDF).
- **fpdf:** Génération de fichiers PDF contenant les résultats de clustering.
- **datetime:** Insertion de la date dans les noms de fichiers exportés.

II K-Means

II.1 Introduction

L'algorithme **K-Means** est une méthode de classification non supervisée très répandue, utilisée pour regrouper des observations en K groupes homogènes appelés *clusters*. Il s'agit d'une méthode de partitionnement qui vise à minimiser l'inertie intra-cluster, c'est-à-dire la variance totale au sein de chaque groupe.

Applications :

- segmentation de marché,
- regroupement de documents,
- compression d'image,
- exploration de données multidimensionnelles.

II.2 Étapes de l'algorithme

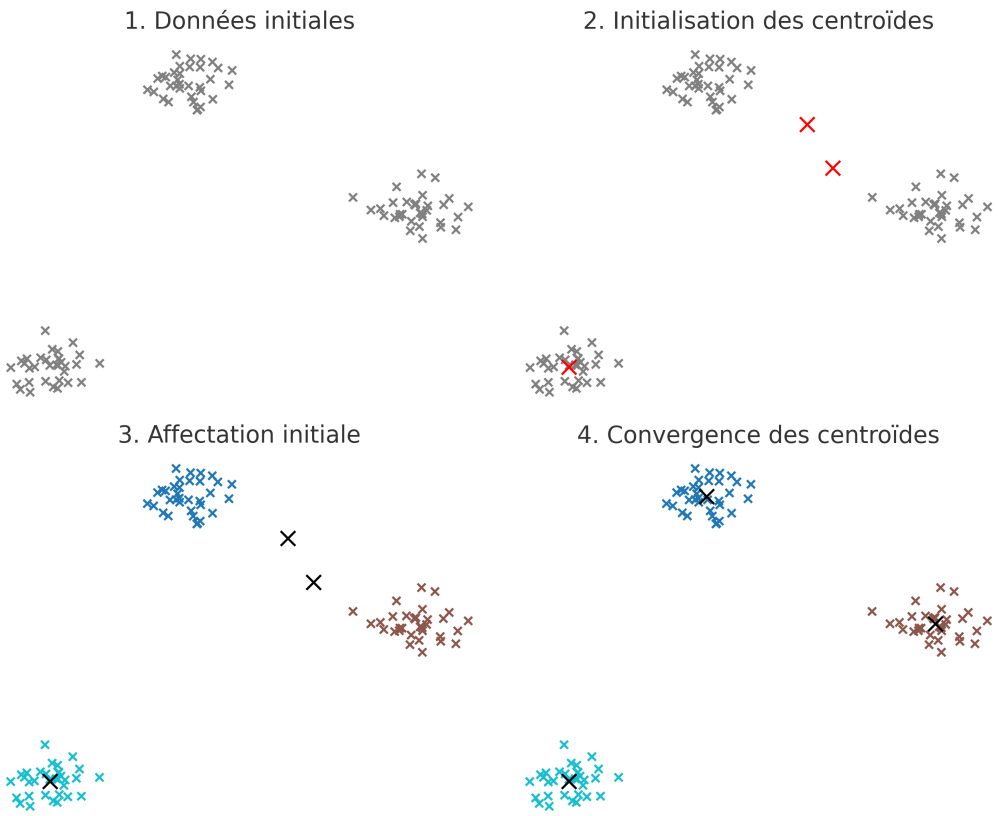


Figure II.1. Étapes du processus de l'algorithme K-Means

Schéma illustratif des étapes de l'algorithme K-means.

Algorithme général

1. **Initialisation** : choisir aléatoirement K centroïdes.
2. **Affectation** : chaque point est assigné au centroïde le plus proche.
3. **Mise à jour** : recalculer les centroïdes en prenant la moyenne des points du cluster.
4. **Convergence** : répéter les deux dernières étapes jusqu'à stabilisation.

II.3 Formulation mathématique

Soit $X = \{x_1, x_2, ..., x_n\}$ un ensemble de n observations en $\mathbb{R}^d$. L'objectif est de trouver une partition $\{C_1, ..., C_K\}$ qui minimise :

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

où :

- K : nombre de clusters,
- C_k : $k^{\text{ème}}$ cluster,
- μ_k : centroïde de C_k, défini par :

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

II.4 Distances possibles

Le choix de la distance influence fortement la forme des clusters :

- **Euclidienne** : $d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$
- **Manhattan** : $d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$
- **Cosine** : $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \cdot \|y\|}$
- **Chebyshev** : $d(x, y) = \max_i |x_i - y_i|$

II.5 Critères de convergence

L'algorithme s'arrête si :

1. les centroïdes ne changent plus entre deux itérations :

$$\mu_k^{(t)} = \mu_k^{(t+1)}$$

2. ou si la variation de l'inertie est inférieure à un seuil ϵ :

$$\frac{|J^{(t+1)} - J^{(t)}|}{J^{(t)}} < \epsilon$$

II.6 Initialisation K-Means++

Pour limiter la dépendance à l'initialisation, K-means++ choisit les centroïdes initiaux de manière plus intelligente :

- Le premier centroïde est sélectionné aléatoirement.
- Les centroïdes suivants sont choisis avec une probabilité proportionnelle à leur distance au plus proche centroïde existant.

II.7 Détermination du nombre optimal de clusters

On utilise souvent la **méthode du coude (elbow method)** :

- Tracer l'inertie J en fonction de K.
- Le point où la courbe commence à "fléchir" indique le bon choix de K.

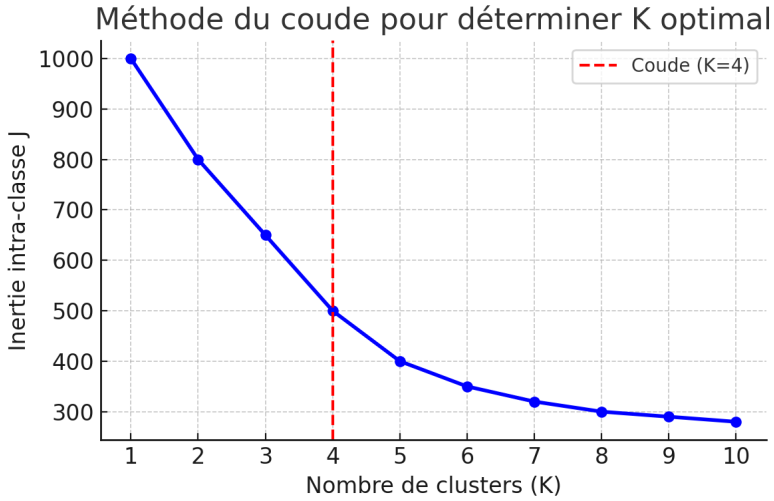


Figure II.2. Méthode du coude pour déterminer le nombre optimal de clusters.

II.8 Avantages

- Simple et rapide sur de grands jeux de données.
- Facile à interpréter.
- Résultats souvent cohérents sur des données sphériques.

II.9 Inconvénients

- Sensible à l'initialisation des centroïdes.
- Suppose que les clusters sont de forme sphérique et de taille équivalente.
- Nécessite de fixer K à l'avance.
- Sensible aux valeurs aberrantes.

III Classification Hiérarchique Ascendante

III.1 Introduction

La **Classification Hiérarchique Ascendante (CAH)** est une méthode de regroupement non supervisée qui permet de construire une hiérarchie de clusters. Contrairement à d'autres méthodes comme le K-means, elle ne nécessite pas de fixer le nombre de clusters au préalable.

La structure hiérarchique obtenue peut être visualisée sous la forme d'un **dendrogramme**, un arbre de regroupement.

III.2 Principe de l'algorithme

L'idée est de :

1. Considérer chaque observation comme un cluster initial.
2. Mesurer la distance entre chaque paire de clusters.
3. Fusionner les deux clusters les plus proches.
4. Répéter jusqu'à obtenir un seul cluster global.

III.3 Description de l'algorithme

La **classification hiérarchique ascendante (CAH)** est une méthode de regroupement par agglomération. Le processus débute avec chaque point de données considéré comme un cluster individuel, et fusionne les clusters par étapes successives jusqu'à ce qu'un seul cluster englobe l'ensemble des données.

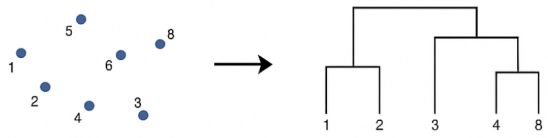


Figure III.1. Illustration du processus de fusion progressive en CAH

Algorithme

- On considère chaque point de données comme un cluster individuel.
- On identifie les deux clusters les plus similaires selon un certain critère de distance.
- On fusionne ces deux clusters pour former un nouveau cluster.
- On répète les étapes précédentes jusqu'à ce que tous les points soient dans un seul cluster.

Formulation mathématique

Soient A et B deux clusters à fusionner, $D(A, B)$ est une mesure de dissimilarité à évaluer.

Types de méthodes de liaison

- **Liaison simple** :
$$D_{\text{simple}}(A, B) = \min_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$
Calculée en minimum entre tous les couples de points des deux clusters.
- **Liaison complète** :
$$D_{\text{complète}}(A, B) = \max_{\substack{i \in A \\ j \in B}} d(x_i, x_j)$$
Calculée en maximum entre tous les couples de points.
- **Liaison moyenne** :
$$D_{\text{moyenne}}(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{j \in B} d(x_i, x_j)$$
Calculée en moyenne entre toutes les paires (x_i, x_j) où $x_i \in A$, $x_j \in B$.
- **Méthode de Ward** :
$$D_{\text{Ward}}(A, B) = \frac{|A||B|}{|A| + |B|} \|\mu_A - \mu_B\|^2$$
Minimise l'augmentation de la variance intra-cluster lors de la fusion.

III.4 Distances entre individus

Les distances utilisées entre points peuvent être :

- **Distance Euclidienne** :
$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$
- **Distance de Manhattan** :
$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
- **Distance de Chebyshev** :
$$d(x, y) = \max_i |x_i - y_i|$$
- **Distance Cosine** :
$$d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$$

III.5 Méthodes de liaison (linkage)

Lors de la fusion de deux clusters A et B, on utilise une méthode de liaison pour définir leur dissimilarité :

- **Liaison simple** :
$$d(A, B) = \min_{x \in A, y \in B} d(x, y)$$
- **Liaison complète** :
$$d(A, B) = \max_{x \in A, y \in B} d(x, y)$$
- **Liaison moyenne** :
$$d(A, B) = \frac{1}{|A||B|} \sum_{x \in A} \sum_{y \in B} d(x, y)$$

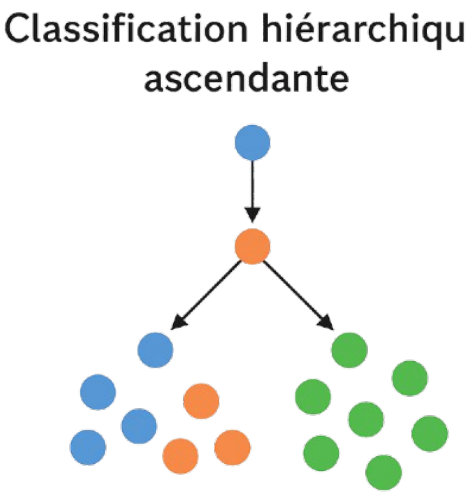
- **Méthode de Ward** :

$$\Delta E(A, B) = \frac{|A||B|}{|A| + |B|} \|\mu_A - \mu_B\|^2$$

  Cette méthode cherche à minimiser l'augmentation de la variance intra-cluster.

III.6 Dendrogramme

Le résultat de la CAH est visualisé à l'aide d'un **dendrogramme**, un arbre de fusions successives. L'axe vertical représente la distance (ou dissimilarité) entre les groupes.



Exemple de dendrogramme

CAH.

III.7 Détermination du nombre de clusters

Il est possible de fixer un nombre de clusters final k en coupant le dendrogramme à une certaine hauteur (distance). On utilise pour cela :

- la **hauteur de fusion**,
- ou des indices de qualité comme la **silhouette**.

III.8 Avantages

- Ne nécessite pas de spécifier k.
- Fournit une vision hiérarchique.
- Compatible avec toutes les distances.

III.9 Inconvénients

- Sensible aux valeurs aberrantes.
- Complexité élevée sur grands jeux de données.
- Les décisions de fusion sont irréversibles.

IV Interface Graphique

IV.1 Page d'Accueil

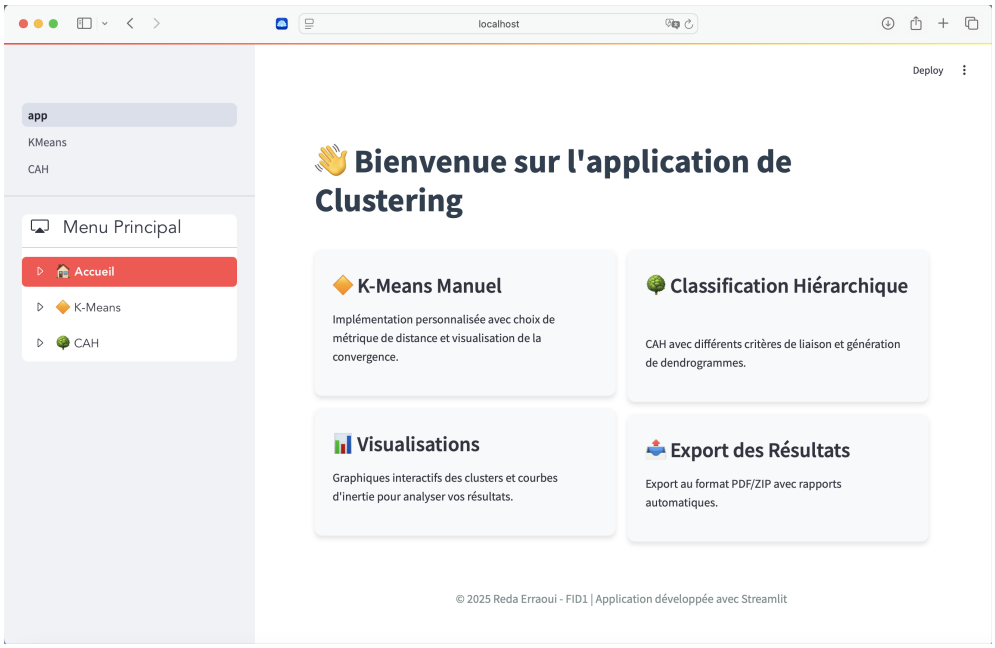


Figure IV.1. Page d'accueil de l'application. L'utilisateur peut importer un fichier Excel ou CSV.

IV.2 Page de Visualisation des Résultats (K-Means)

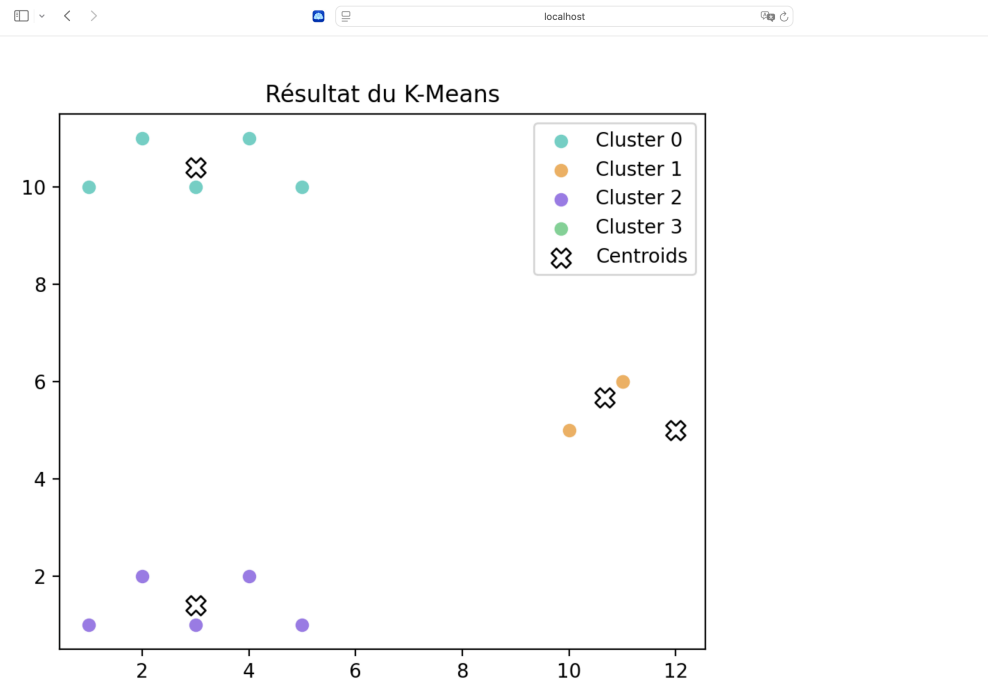


Figure IV.2. Visualisation des clusters obtenus avec l'algorithme K-Means.

IV.3 Courbe de convergence

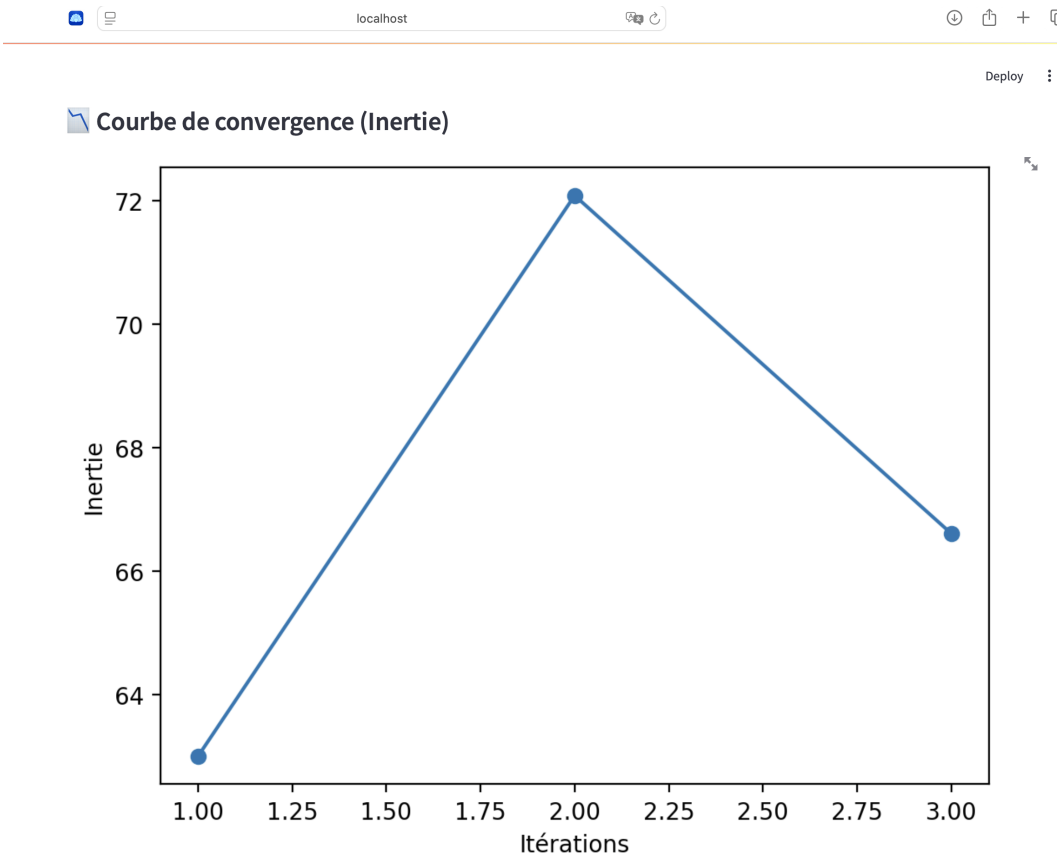


Figure IV.3. Affichage de la courbe de convergence montrant l'inertie au fil des itérations.

IV.4 Visualisation Hiérarchique (CAH)

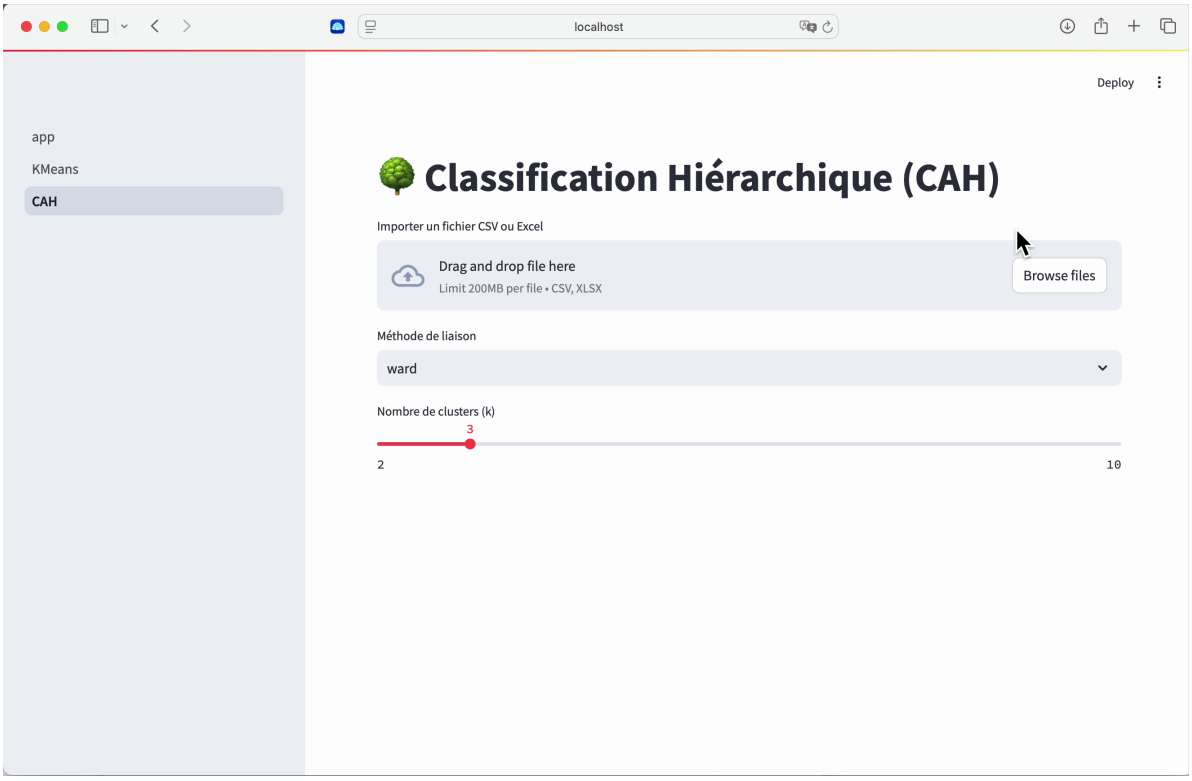


Figure IV.4. Dendrogramme illustrant les fusions successives entre clusters (CAH).

IV.5 Page de Visualisation du Dendogramme

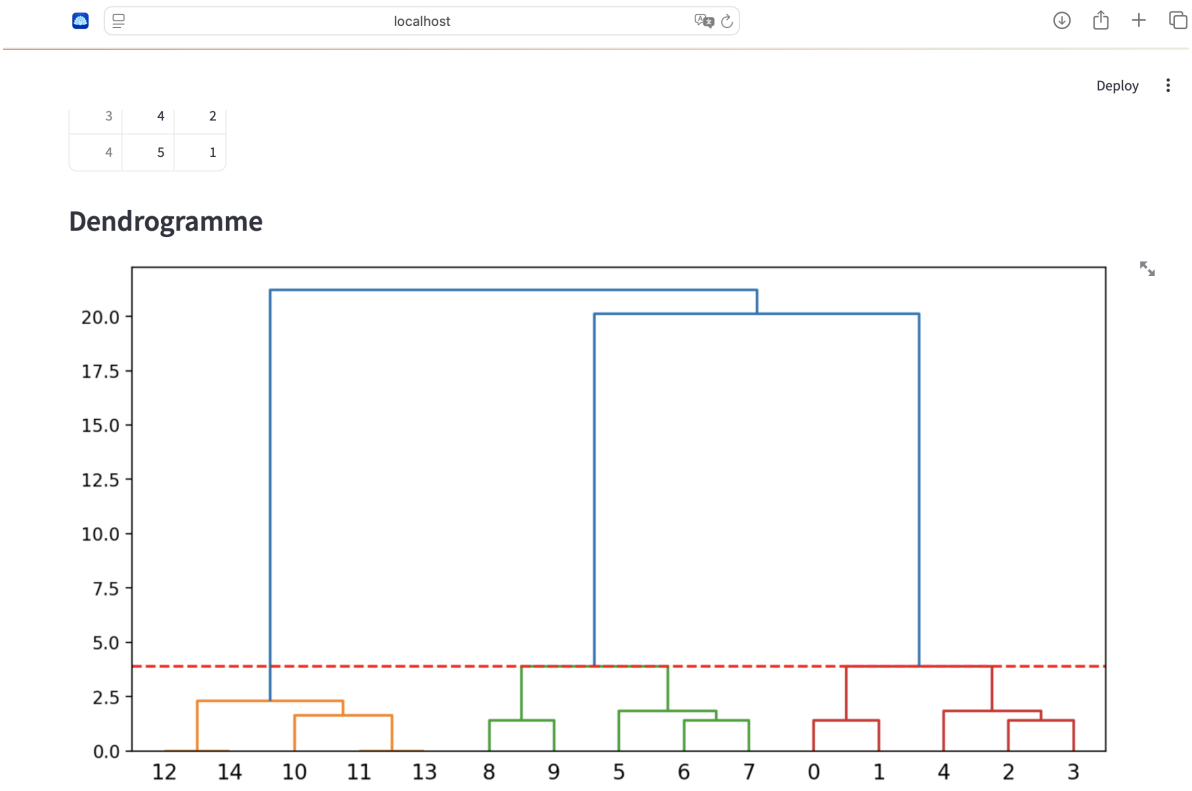


Figure IV.5. Options d'exportation vers PDF ou ZIP des graphiques et logs.

IV.6 Export des Résultats

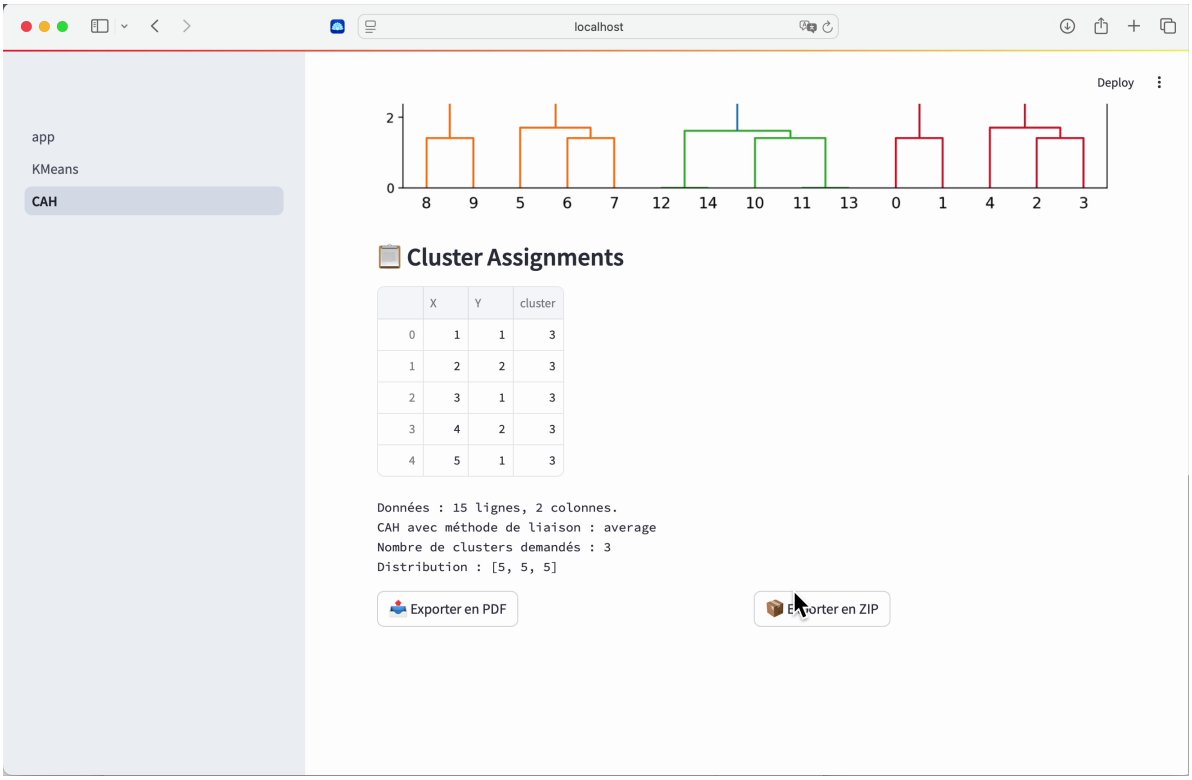


Figure IV.6. Page d'export des résultats avec options PDF et ZIP.

V Conclusion

Au terme de ce projet, nous avons mis en œuvre deux méthodes fondamentales de classification non supervisée : **K-Means** et la **Classification Hiérarchique Ascendante (CAH)**. Ces deux approches, bien que différentes dans leur principe, se sont révélées complémentaires pour explorer et regrouper des données multivariées.

L'algorithme **K-Means** se distingue par sa simplicité, sa rapidité d'exécution et son efficacité sur de grands jeux de données. Il est particulièrement adapté aux structures sphériques et bien séparées. Son principal inconvénient réside dans la nécessité de spécifier le nombre de clusters à l'avance et sa sensibilité à l'initialisation.

À l'inverse, la **CAH** offre une approche plus visuelle grâce au dendrogramme, permettant d'explorer la structure hiérarchique des données sans présupposer le nombre de groupes. Elle se révèle plus souple dans les contextes exploratoires, bien qu'elle soit plus coûteuse en termes de complexité algorithmique.

L'intégration de ces méthodes dans une **application interactive** a permis de tester leur comportement sur divers jeux de données, avec une visualisation directe des résultats, des distances, des liaisons et de la convergence. L'interface graphique développée facilite la manipulation des données, l'affichage dynamique des clusters, l'export des résultats, ainsi que l'analyse comparative entre K-Means et CAH.

Ce projet nous a permis de :

- Maîtriser les aspects mathématiques et algorithmiques du clustering.
- Implémenter ces méthodes de manière manuelle pour une compréhension approfondie.
- Créer une application conviviale et fonctionnelle à l'aide de bibliothèques Python.
- Mieux comprendre les forces, les limites et les domaines d'application de chaque méthode.

Ce travail constitue une base solide pour des analyses plus avancées, telles que la classification fuzzy, DBSCAN ou l'utilisation combinée de méthodes supervisées et non supervisées.

VI Annexe

1. Interface d'accueil de l'application Streamlit (app.py):

```
import streamlit as st
from streamlit_option_menu import option_menu

# Configuration de la page
st.set_page_config(
page_title="Clustering App",
layout="wide",
page_icon=" "
)

# Style CSS commun
COMMON_STYLE = """
<style>
.header { color: #2c3e50; margin-bottom: 1.5rem; }
.feature-card {
    background-color: #f8f9fa;
    border-radius: 10px;
    padding: 1.5rem;
    margin-bottom: 1rem;
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
.upload-box { border: 2px dashed #ccc; border-radius: 5px; padding:
   2rem; text-align: center; }
.footer { text-align: center; margin-top: 3rem; color: #7f8c8d; }
.plot-container { background-color: white; padding: 1rem;
   border-radius: 5px; }
</style>
"""
st.markdown(COMMON_STYLE, unsafe_allow_html=True)

# Navigation
with st.sidebar:
selected = option_menu(
"Menu Principal",
[" Accueil", " K-Means", " CAH"],
menu_icon="cast",
default_index=0,
styles={
    "container": {"padding": "0!important"},
    "nav-link": {"font-size": "16px", "text-align": "left"}
}
)

if selected == " Accueil":
st.markdown('<h1 class="header"> Bienvenue sur l\'application de
   Clustering</h1>', unsafe_allow_html=True)

col1, col2 = st.columns(2)
with col1:
st.markdown("""
<div class="feature-card">
```

```
        <h3> K-Means Manuel</h3>
        <p>Impl\'ementation personnalis\'ee avec choix de m\'etrique de
           distance et visualisation de la convergence.</p>
        </div>
        """, unsafe_allow_html=True)

        st.markdown("""
        <div class="feature-card">
        <h3> Visualisations</h3>
        <p>Graphiques interactifs des clusters et courbes d'inertie pour
           analyser vos r\'esultats.</p>
        </div>
        """, unsafe_allow_html=True)

        with col2:
        st.markdown("""
        <div class="feature-card">
        <h3> Classification Hi\'erarchique</h3>
        <p>CAH avec diff rents crit\ res  de liaison et g n ration de
           dendrogrammes.</p>
        </div>
        """, unsafe_allow_html=True)

        st.markdown("""
        <div class="feature-card">
        <h3> Export des R sultats</h3>
        <p>Export au format PDF/ZIP avec rapports automatiques.</p>
        </div>
        """, unsafe_allow_html=True)

        st.markdown('<div class="footer">  2025 Reda Erraoui - FID1 |
           Application d velopp e avec Streamlit</div>',
           unsafe_allow_html=True)

        elif selected == " K-Means":
        st.switch_page("pages/1_KMeans.py")
        elif selected == " CAH":
        st.switch_page("pages/2_CAH.py")
```

2. Page K-Means manuelle (kmeans_manual.py):

```
        import streamlit as st
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        from io import BytesIO

        st.set_page_config(layout="wide")
        st.title("     K-Means Clustering")

        # Impl mentation KMeans (version nettoy e)
        class KMeans:
        def __init__(self, k=3, distance_metric='euclidean',
           random_state=None):
        self.k = k
        self.distance_metric = distance_metric
        self.random_state = random_state
        self.inertia_history = []

        def fit(self, X, max_iterations=200):
```

```
        np.random.seed(self.random_state)
        self.centroids = self._initialize_centroids(X)

        for _ in range(max_iterations):
        distances = self._calculate_distances(X)
        clusters = np.argmin(distances, axis=0)
        new_centroids = np.array([X[clusters == i].mean(axis=0) for i in
            range(self.k)])

        if np.allclose(self.centroids, new_centroids):
        break

        self.centroids = new_centroids
        self.inertia_history.append(np.sum(np.min(distances, axis=0)))

        return clusters, self.centroids, self.inertia_history

        def _calculate_distances(self, X):
        if self.distance_metric == 'euclidean':
        return np.array([np.linalg.norm(X - c, axis=1) for c in
            self.centroids])
        elif self.distance_metric == 'city_block':
        return np.array([np.sum(np.abs(X - c), axis=1) for c in
            self.centroids])
        else:
        raise ValueError("M trique de distance non support e")

        def _initialize_centroids(self, X):
        indices = np.random.choice(len(X), self.k, replace=False)
        return X[indices]

        # Interface utilisateur
        with st.expander("     Importation des donn es", expanded=True):
        uploaded_file = st.file_uploader("Choisir un fichier CSV/Excel",
            type=["csv", "xlsx"])

        col1, col2 = st.columns(2)
        with col1:
        k_value = st.slider("Nombre de clusters (k)", 2, 10, 3)
        metric = st.selectbox("M trique de distance", ["euclidean",
            "city_block"])
        random_seed = st.number_input("Random seed", value=42)

        with col2:
        max_iter = st.slider("Nombre max d'it rations", 10, 500, 100)
        show_history = st.checkbox("Afficher l'historique de convergence",
            True)

        if uploaded_file:
        try:
        df = pd.read_csv(uploaded_file) if
            uploaded_file.name.endswith('.csv') else
            pd.read_excel(uploaded_file)

        st.subheader("Aper u des donn es")
        st.dataframe(df.head())

        X = df.select_dtypes(include=np.number).values

        if X.shape[1] < 2:
        st.error("Les donn es doivent contenir au moins 2 colonnes
            num riques")
        else:
```

```
        model = KMeans(k=k_value, distance_metric=metric,
            random_state=random_seed)
        clusters, centroids, history = model.fit(X, max_iterations=max_iter)

        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

        # Visualisation clusters
        ax1.scatter(X[:, 0], X[:, 1], c=clusters, cmap='viridis', alpha=0.5)
        ax1.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X',
            s=200)
        ax1.set_title("Visualisation des Clusters")

        # Courbe de convergence
        if show_history and len(history) > 0:
        ax2.plot(range(len(history)), history, marker='o')
        ax2.set_title("Courbe de Convergence")
        ax2.set_xlabel("It rations")
        ax2.set_ylabel("Inertie")

        st.pyplot(fig)

        except Exception as e:
        st.error(f"Erreur lors du traitement: {str(e)}")
        else:
        st.info("Veuillez uploader un fichier CSV ou Excel pour commencer
            l'analyse")
```

3. Fonctions d'export PDF et ZIP (export_utils.py)

```
        import os
        import zipfile
        from fpdf import FPDF

        def export_pdf(title, log_lines, image_path, output_path):
        pdf = FPDF()
        pdf.add_page()
        pdf.set_font("Arial", size=12)
        pdf.cell(200, 10, txt=title, ln=True, align="C")
        pdf.ln(10)

        for line in log_lines:
        pdf.multi_cell(0, 10, txt=line)
        pdf.ln(5)

        if os.path.exists(image_path):
        pdf.image(image_path, x=10, w=180)

        pdf.output(output_path)

        def export_zip(log_txt, csv_path, image_path, zip_path):
        with zipfile.ZipFile(zip_path, 'w') as zipf:
        zipf.writestr("log.txt", log_txt)
        if os.path.exists(csv_path):
        zipf.write(csv_path, arcname=os.path.basename(csv_path))
        if os.path.exists(image_path):
        zipf.write(image_path, arcname=os.path.basename(image_path))
```

4. Page K-Means avec export PDF/ZIP (1_KMeans.py)

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from modules.kmeans_manual import KMeans
from modules.export_utils import export_pdf, export_zip
import os

st.title("     K-Means Clustering Manuel")

uploaded_file = st.file_uploader("Importer un fichier CSV ou Excel",
    type=["csv", "xlsx"])
distance = st.selectbox("Distance utilis e", ["euclidean",
    "city_block"])
k = st.slider("Nombre de clusters (k)", 2, 10, 3)

if uploaded_file:
if uploaded_file.name.endswith(".csv"):
df = pd.read_csv(uploaded_file)
else:
df = pd.read_excel(uploaded_file)

st.subheader("Aper u des donn es")
st.dataframe(df.head())

X = df.values
model = KMeans(k=k, distance_metric=distance)
labels, centroids, inertias = model.fit(X)

st.subheader("     R sultat du Clustering")
fig, ax = plt.subplots(figsize=(6, 5))
cluster_colors = ['#4FD1C5', '#F6AD55', '#9F7AEA', '#68D391',
    '#F56565']
for i in np.unique(labels):
ax.scatter(X[labels == i, 0], X[labels == i, 1],
label=f"Cluster {i}", color=cluster_colors[i % len(cluster_colors)])
ax.scatter(centroids[:, 0], centroids[:, 1], c='white',
    edgecolors='black',
marker='X', s=100, label='Centroids')
ax.set_title("R sultat du K-Means")
ax.legend()
st.pyplot(fig)

# Cluster assignments
df_clusters = df.copy()
df_clusters["cluster"] = labels
st.subheader("     Cluster Assignments")
st.dataframe(df_clusters.head())

# Courbe de convergence
st.subheader("     Courbe de convergence (Inertie)")
fig2, ax2 = plt.subplots()
ax2.plot(range(1, len(inertias) + 1), inertias, marker='o')
ax2.set_xlabel("It rations")
ax2.set_ylabel("Inertie")
st.pyplot(fig2)

# Log
log_lines = [
f"Donn es : {X.shape[0]} lignes, {X.shape[1]} colonnes.",
f"K-Means avec distance : {distance}",
```

```
        f"Nombre de clusters : {k}",
        f"Inertie finale : {inertias[-1]:.2f}",
        f"Distribution : {[list(labels).count(i) for i in range(k)]}"
        ]
        st.text("\n".join(log_lines))

        # Sauvegarde
        os.makedirs("results", exist_ok=True)
        fig.savefig("results/kmeans_result.png")
        df_clusters.to_csv("results/kmeans_clusters.csv", index=False)

        # Boutons export
        col1, col2 = st.columns(2)
        with col1:
        if st.button("     Exporter en PDF"):
        export_pdf("R sultat du K-Means", log_lines,
            "results/kmeans_result.png", "results/kmeans_result.pdf")
        with open("results/kmeans_result.pdf", "rb") as f:
        st.download_button("T l charger le PDF", f,
            file_name="result_kmeans.pdf")

        with col2:
        if st.button("     Exporter en ZIP"):
        export_zip("\n".join(log_lines), "results/kmeans_clusters.csv",
            "results/kmeans_result.png", "results/kmeans_result.zip")
        with open("results/kmeans_result.zip", "rb") as f:
        st.download_button("T l charger le ZIP", f,
            file_name="result_kmeans.zip")
```

5. Page CAH avec export PDF/ZIP (2_CAH.py)

```
        import streamlit as st
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
        import os
        from modules.export_utils import export_pdf, export_zip

        st.title("     Classification Hi rarchique (CAH)")

        uploaded_file = st.file_uploader("Importer un fichier CSV ou Excel",
            type=["csv", "xlsx"])
        linkage_method = st.selectbox("M thode de liaison", ["ward",
            "single", "complete", "average"])
        k = st.slider("Nombre de clusters (k)", 2, 10, 3)

        if uploaded_file:
        if uploaded_file.name.endswith(".csv"):
        df = pd.read_csv(uploaded_file)
        else:
        df = pd.read_excel(uploaded_file)

        st.subheader("Aper u des donn es")
        st.dataframe(df.head())

        X = df.values
        Z = linkage(X, method=linkage_method)

        st.subheader("Dendrogramme")
        fig, ax = plt.subplots(figsize=(10, 5))
```

```
        dendrogram(Z, ax=ax)
        ax.axhline(y=Z[-k, 2], color='red', linestyle='--')
        st.pyplot(fig)

        # D coupage en clusters
        labels = fcluster(Z, k, criterion='maxclust')
        df_clusters = df.copy()
        df_clusters["cluster"] = labels

        st.subheader("     Cluster Assignments")
        st.dataframe(df_clusters.head())

        # Log
        log_lines = [
        f"Donn es : {X.shape[0]} lignes, {X.shape[1]} colonnes.",
        f"CAH avec m thode de liaison : {linkage_method}",
        f"Nombre de clusters demand s : {k}",
        f"Distribution : {[list(labels).count(i) for i in range(1, k+1)]}"
        ]
        st.text("\n".join(log_lines))

        # Sauvegarde des fichiers
        img_path = "results/dendrogram_cah.png"
        csv_path = "results/clusters_cah.csv"
        pdf_path = "results/result_cah.pdf"
        zip_path = "results/CAH_result.zip"

        os.makedirs("results", exist_ok=True)
        fig.savefig(img_path)
        df_clusters.to_csv(csv_path, index=False)

        # Boutons d'export
        col1, col2 = st.columns(2)
        with col1:
        if st.button("     Exporter en PDF"):
        export_pdf("R sultat du CAH", log_lines, img_path, pdf_path)
        st.success("PDF g  n  r   avec succ s.")
        with open(pdf_path, "rb") as f:
        st.download_button("T l charger le PDF", f,
           file_name="result_cah.pdf")

        with col2:
        if st.button("     Exporter en ZIP"):
        export_zip("\n".join(log_lines), csv_path, img_path, zip_path)
        st.success("ZIP  g  n  r   avec succ s.")
        with open(zip_path, "rb") as f:
        st.download_button("T l charger le ZIP", f,
           file_name="result_cah.zip")
```